VIT Vellore

School of Computer Science and Engineering (SCOPE)

Project Report

Customer Management System

Course Code	: MACSE639
Course Title	: DevOps
Course Type	: ETH + ELA
Slot	: D1 + D2 / L9+L10, L39+L40
School	: SCOPE
Faculty	: SUDHAKAR. P B
Total Marks	: 50
Semester	: Winter Semester 2025–2026
Set	: B

Group 10

Team Members

Aryalekshmi S	25MCS0059
Lavanya Singh	25MCS0079
Edamalapati Saranya	25MCS0096

30th March 2026

0.1 Step 1 – Create Java Project and All Classes

A Maven project named `customer-management` was created in NetBeans IDE with group ID `com.customermgmt` and version `1.0.0`. All classes were created under the `customermgmt` package.

0.1.1 Customer.java

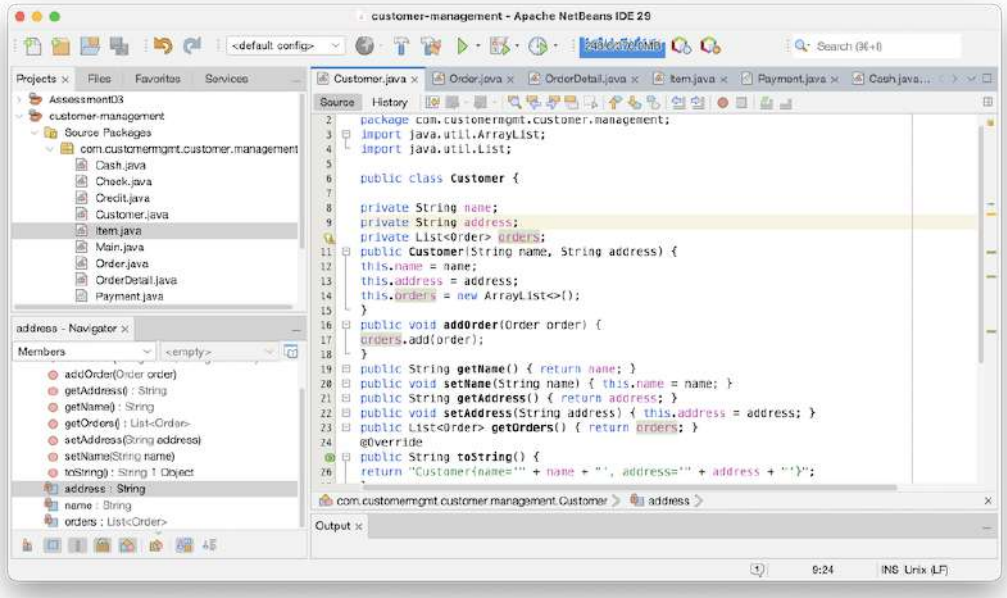

Figure 1: NetBeans editor showing `Customer.java` – contains `name`, `address` fields and an `orders` list with `addOrder()` method

0.1.2 Order.java

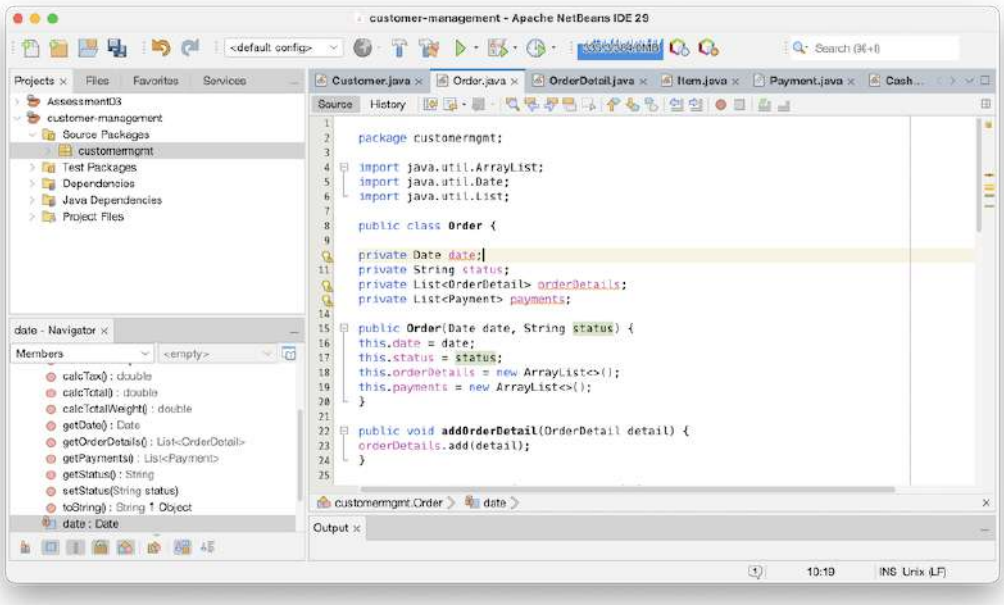

Figure 2: NetBeans editor showing `Order.java` – implements `calcSubTotal()`, `calcTax()`, `calcTotal()`, and `calcTotalWeight()` by iterating over `OrderDetail` list

0.1.3 OrderDetail.java

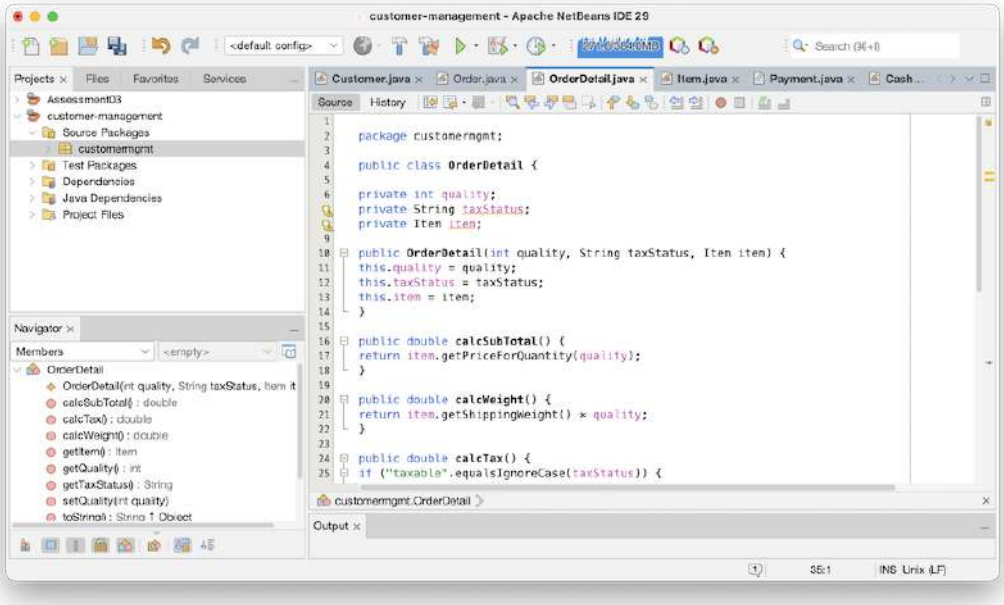

Figure 3: NetBeans editor showing `OrderDetail.java` – links an `Item` with quantity and tax status; implements `calcSubTotal()`, `calcWeight()`, `calcTax()`

0.1.4 Item.java

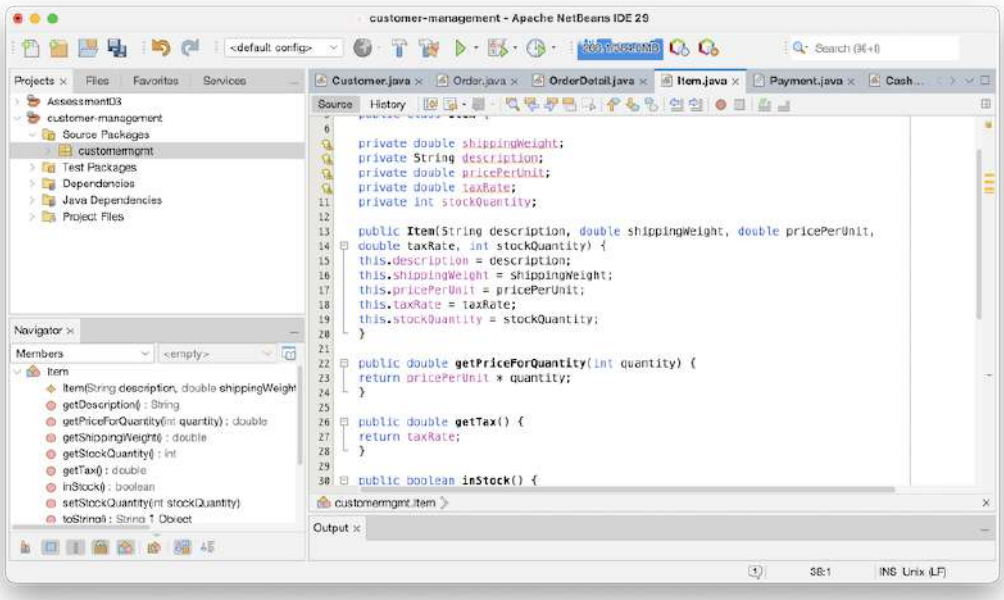

Figure 4: NetBeans editor showing `Item.java` – contains `shippingWeight`, `description`, `pricePerUnit`, `taxRate`, `stockQuantity` with `getPriceForQuantity()`, `getTax()`, `inStock()`

0.1.5 Payment.java (Abstract)

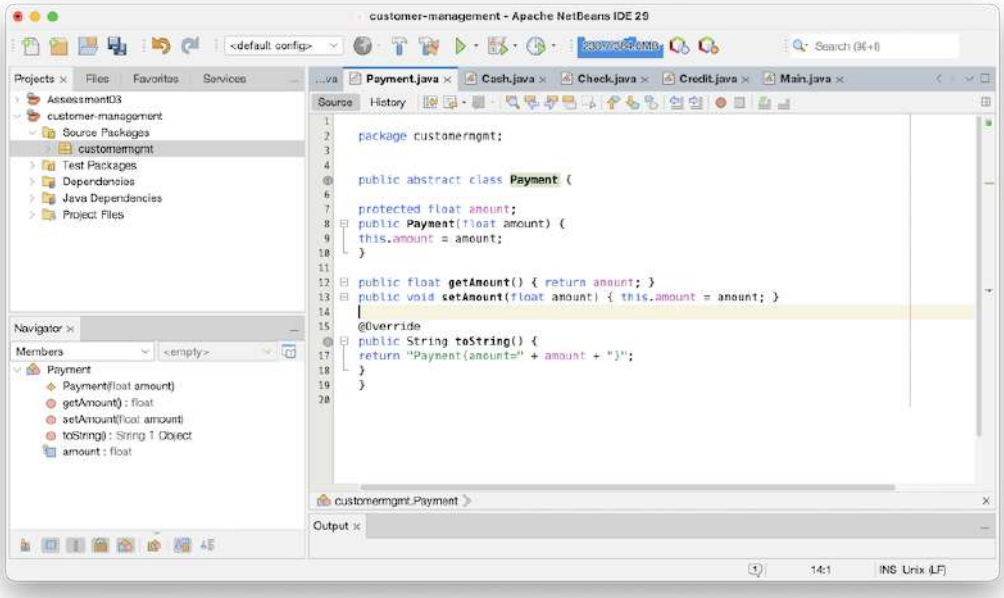

Figure 5: NetBeans editor showing `Payment.java` – abstract base class with `amount` field; never instantiated directly, only through subclasses Cash, Check, Credit

0.1.6 Cash.java

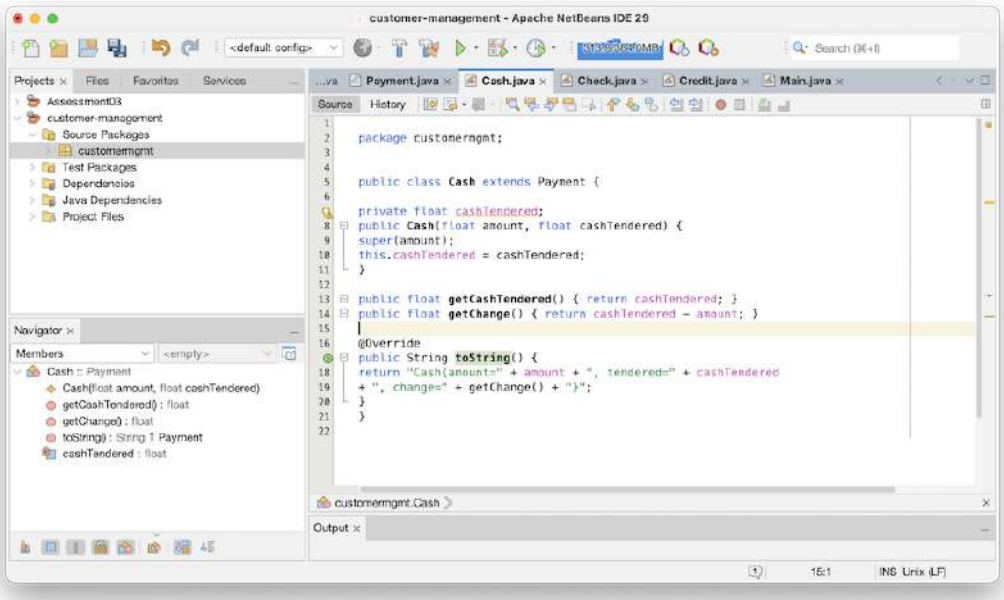

Figure 6: NetBeans editor showing `Cash.java` – extends `Payment` with `cashTendered` field and `getChange()` method computing change returned to customer

0.1.7 Check.java

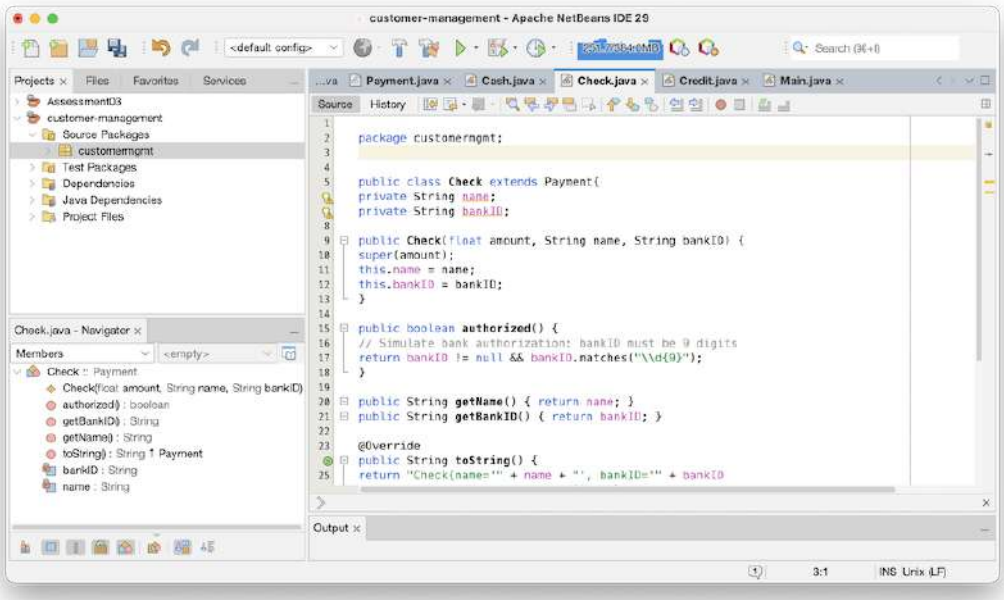

Figure 7: NetBeans editor showing `Check.java` – extends `Payment` with `name` and `bankID`; `authorized()` validates that the bank ID is exactly 9 digits

0.1.8 Credit.java

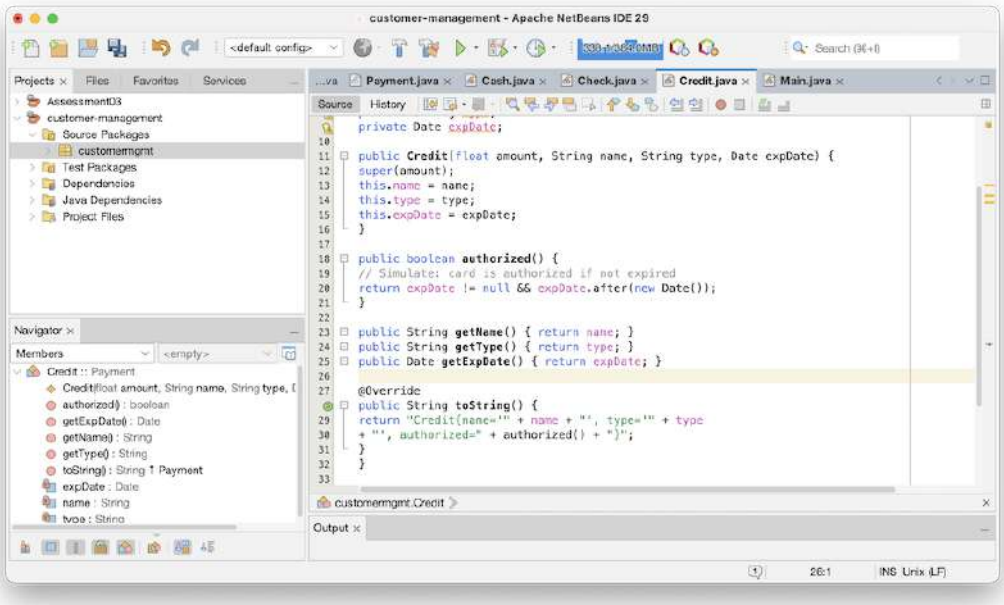

Figure 8: NetBeans editor showing `Credit.java` – extends `Payment` with `name`, `type`, `expDate`; `authorized()` checks the expiry date is in the future

0.1.9 Main.java

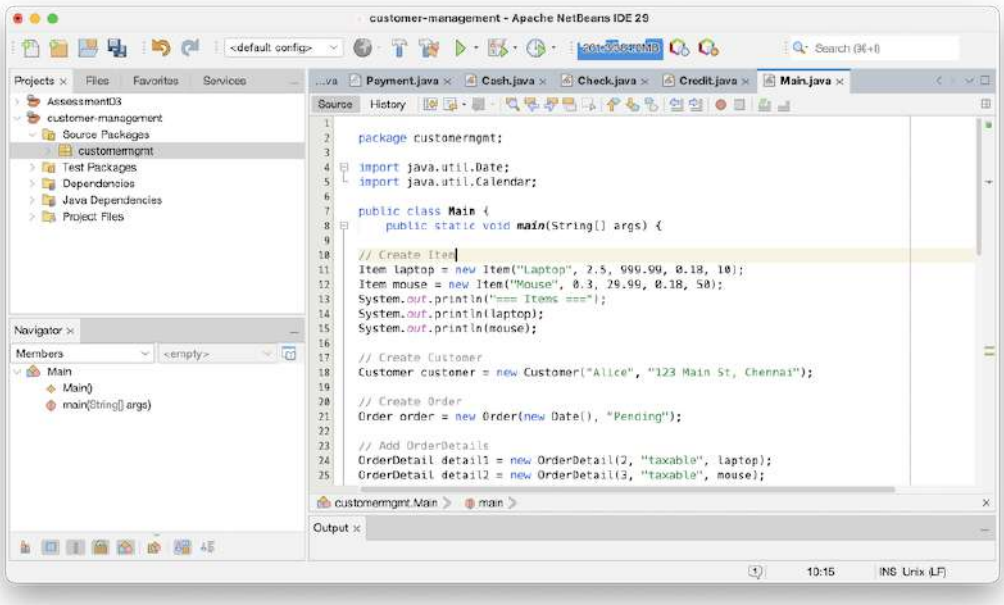

Figure 9: NetBeans editor showing `Main.java` – driver class creating sample `Item`, `Customer`, `Order`, `OrderDetail` and all three payment types, then printing results

0.1.10 Build and Run Output

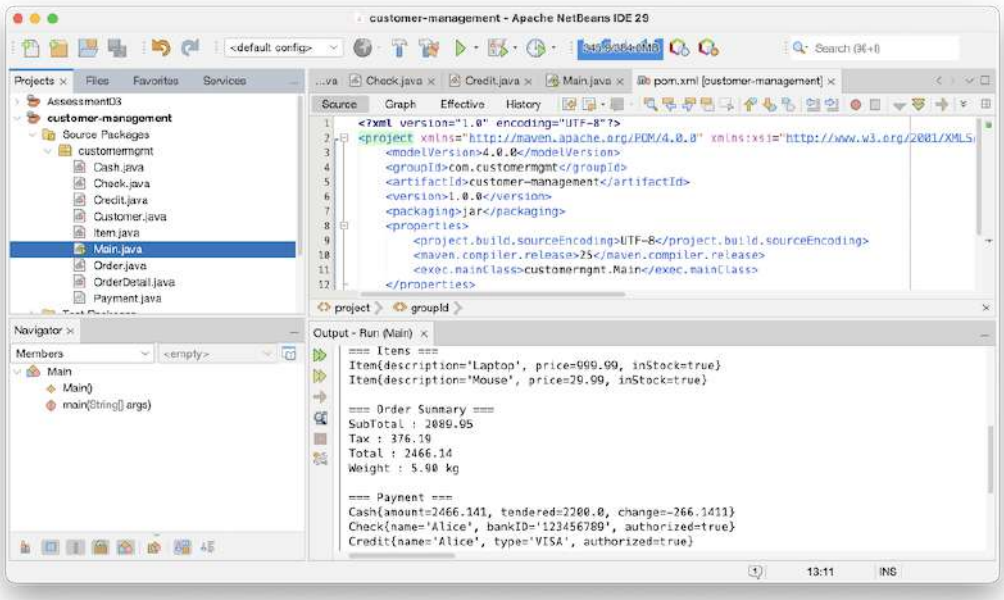

Figure 10: NetBeans Output panel showing successful build and run of `Main.java` – displays Items, Order Summary (SubTotal, Tax, Total, Weight), Payment details, and Customer information

The output confirms all classes interact correctly. The order total, tax calculations, and payment authorization work as expected from the class diagram.

1 Question 2: Source Code Repository (Git/GitHub)

1.1 Overview

Git is used for version control with GitHub as the remote repository. We follow the **Git Flow** branching strategy where each team member develops on a dedicated feature branch and merges via Pull Request into `develop`, which is finally merged into `main` for releases.

1.2 Branching Strategy

Branch	Purpose
`main`	Production-ready code only; protected branch
`develop`	Integration branch; all feature branches merge here
`feature/customer-model`	Member 1 – Customer.java
`feature/order-model`	Member 2 – Order.java, OrderDetail.java
`feature/item-model`	Member 3 – Item.java
`feature/payment-model`	Member 4 – Payment, Cash, Check, Credit
`feature/unit-tests`	Member 5 – CustomerManagementTest.java

Table 1: Git Flow Branching Structure

1.3 Version Tags

Tag	Description
`v1.0.0`	First stable release – all classes complete, CI green

Table 2: Semantic Version Tag

1.4 Implementation Steps

1.4.1 Step 1 – Create GitHub Repository

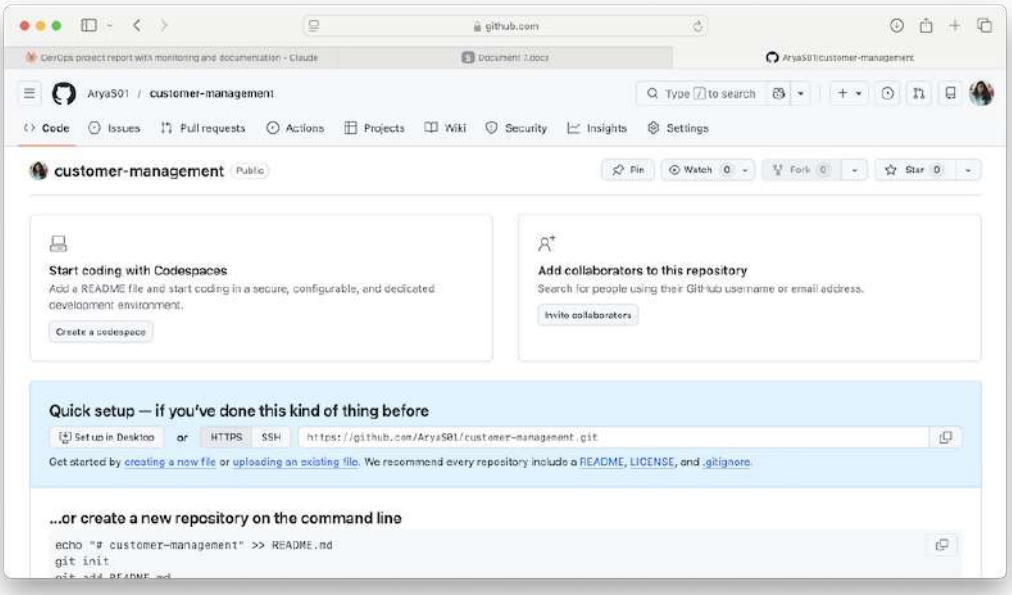

Figure 11: GitHub.com showing the newly created `customer-management` repository with repository name, description, and visibility settings

1.4.2 Step 2 – Initialize Git in NetBeans

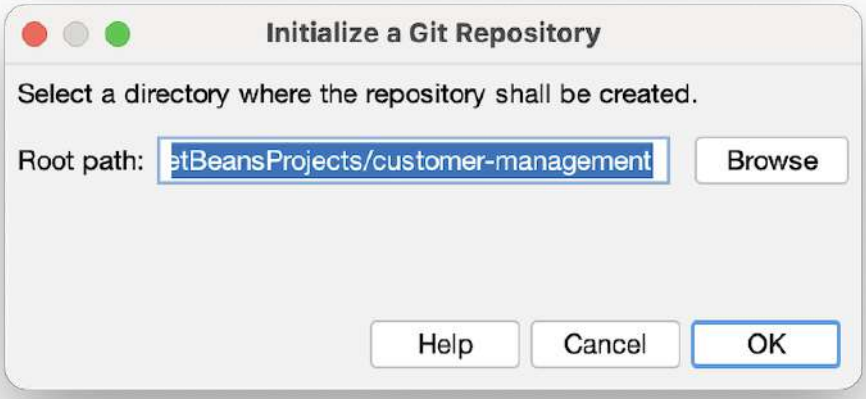

Figure 12: NetBeans *Team → Git → Initialize Repository* dialog showing the project folder path selected for Git initialisation; files turn blue/green indicating tracking has begun

1.4.3 Step 3 – Create .gitignore

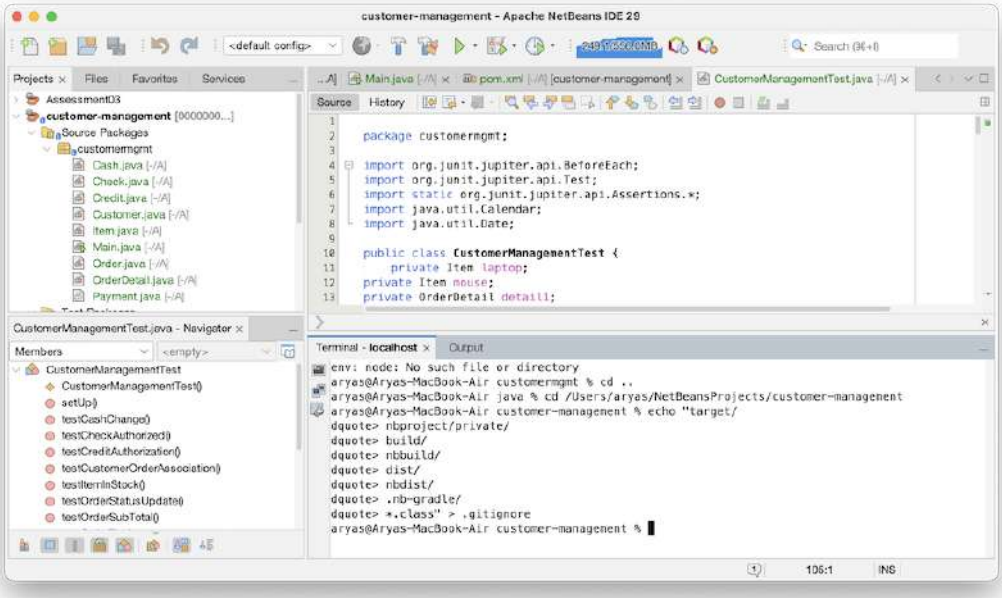

Figure 13: Terminal window showing creation of `.gitignore` file, excluding `target/`, `nbproject/private/`, `*.class` and other build artefacts from version control

1.4.4 Step 4 – First Commit on main

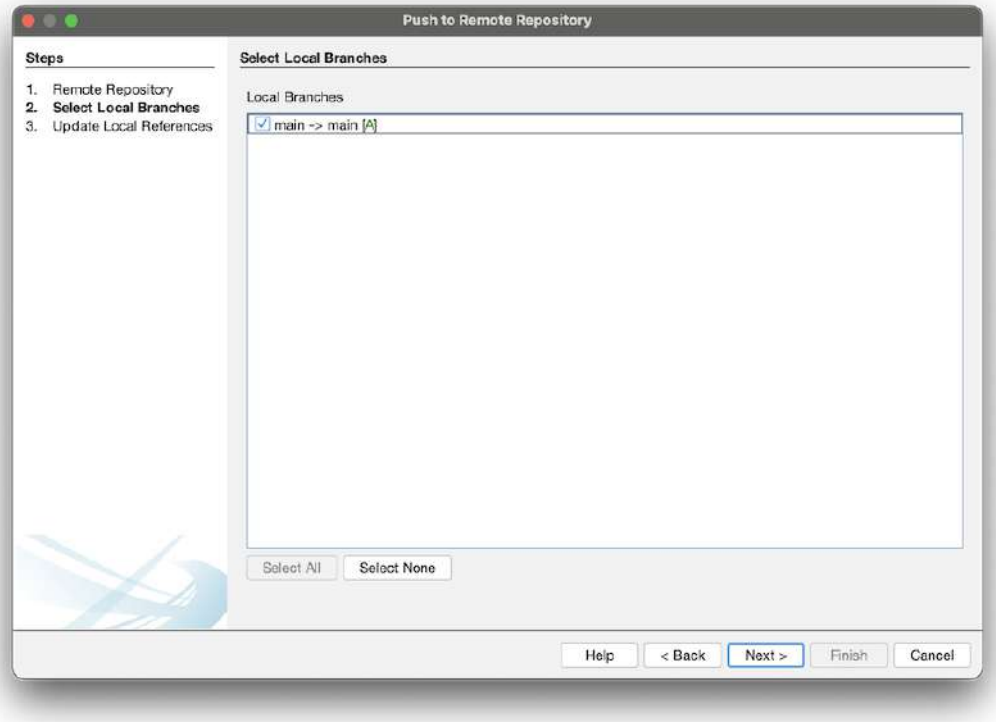

Figure 14: NetBeans Git Commit dialog showing all project files selected (staged) with commit message `feat: initial Customer Management prototype`; files listed in red indicating untracked status before staging

1.4.5 Step 5 – Create develop Branch

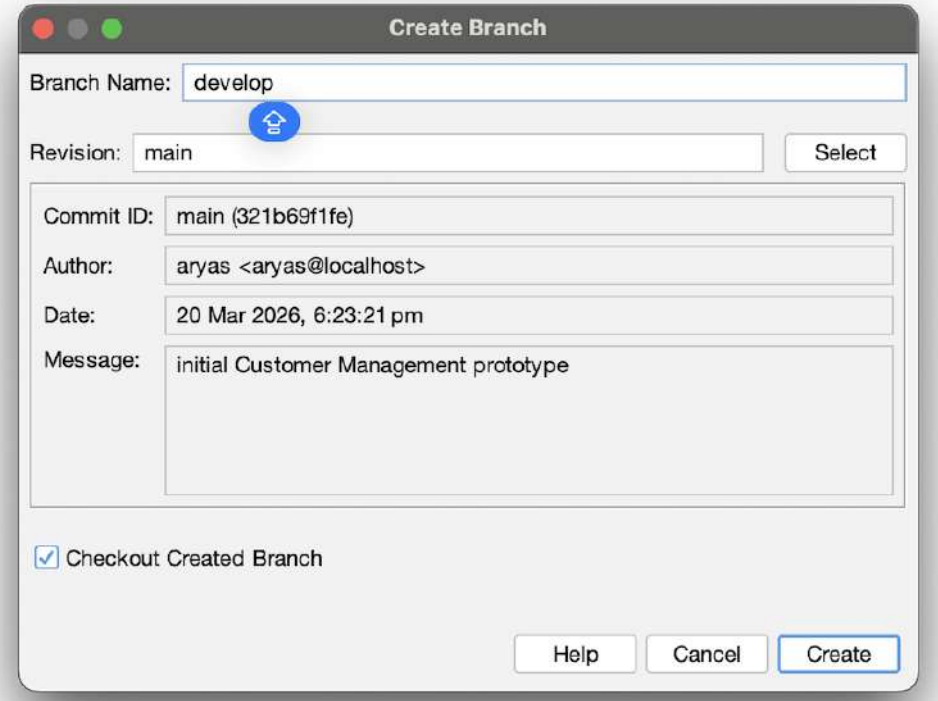

Figure 15: NetBeans *Create Branch* dialog showing `develop` branch being created from `main` with *Checkout Branch* option checked, switching the working directory to the new branch

1.4.6 Step 6 – Push to develop Branch

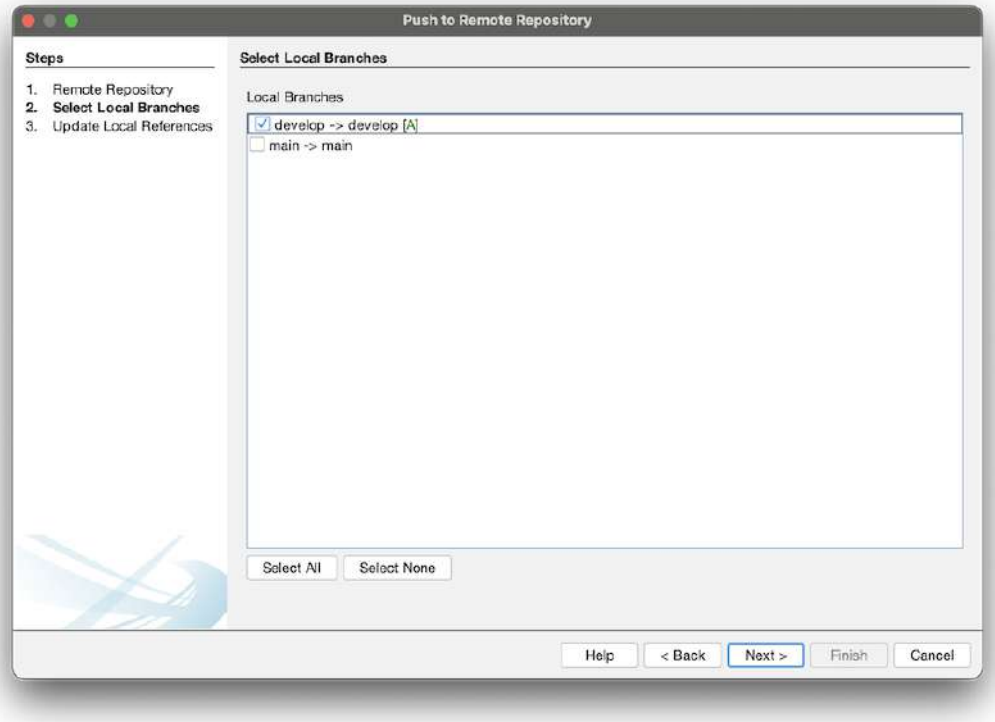

Figure 16: NetBeans *Push to Remote Repository* dialog showing `develop` branch selected and mapped to remote `origin/develop` on GitHub

1.4.7 Step 7 – Create Feature Branches

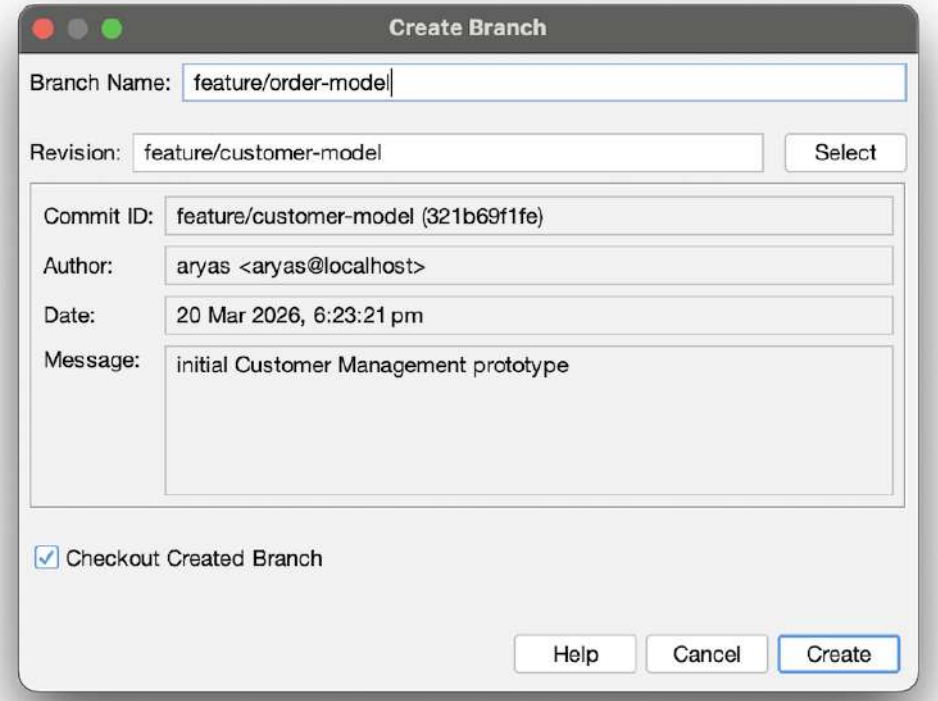

Figure 17: NetBeans *Create Branch* dialog showing `feature/customer-model` branch being created from `develop`; similar steps were repeated for all five feature branches

1.4.8 Step 8 – Commit and Push All Branches

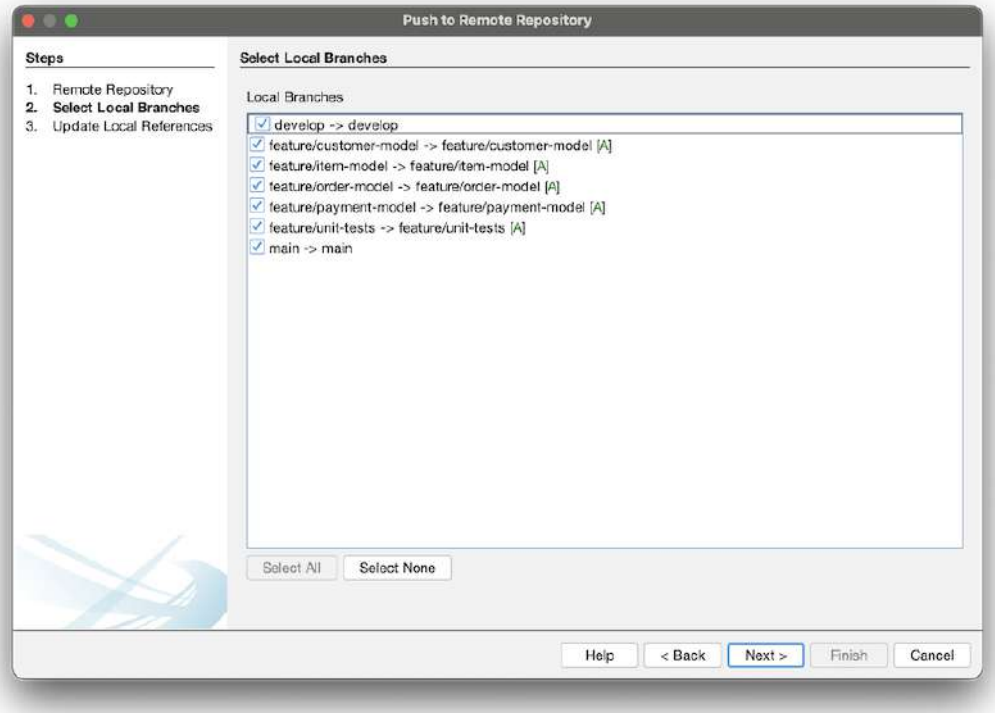

Figure 18: NetBeans *Push to Remote Repository* dialog showing all branches selected – `develop`, `feature/customer-model`, `feature/item-model`, `feature/order-model`, `feature/payment-model`, `feature/unit-tests`, and `main` – being pushed to GitHub simultaneously

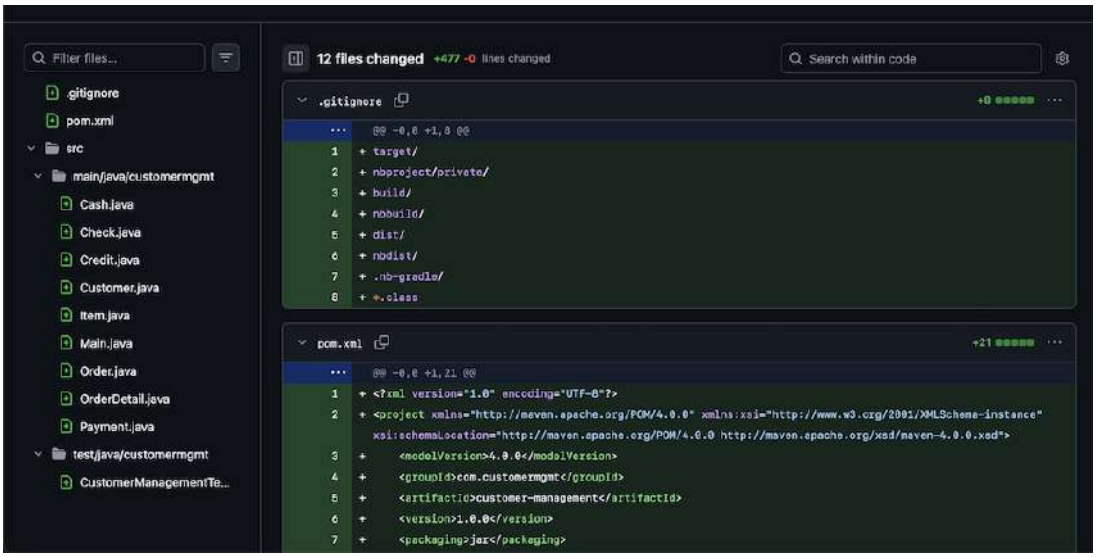

Figure 19: Github output panel confirming successful push of all branches to the remote repository `origin`

1.4.9 Step 9 – Raise a Pull Request on GitHub

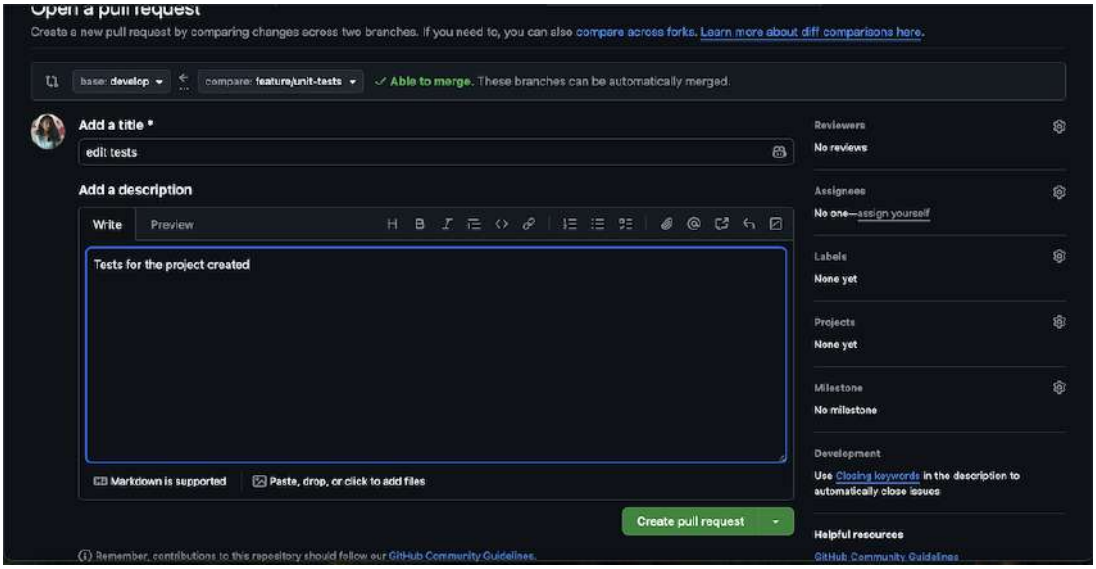

Figure 20: GitHub Pull Request creation page showing `feature/unit-test` branch being merged into `develop`, with title, description, and reviewer assignment fields

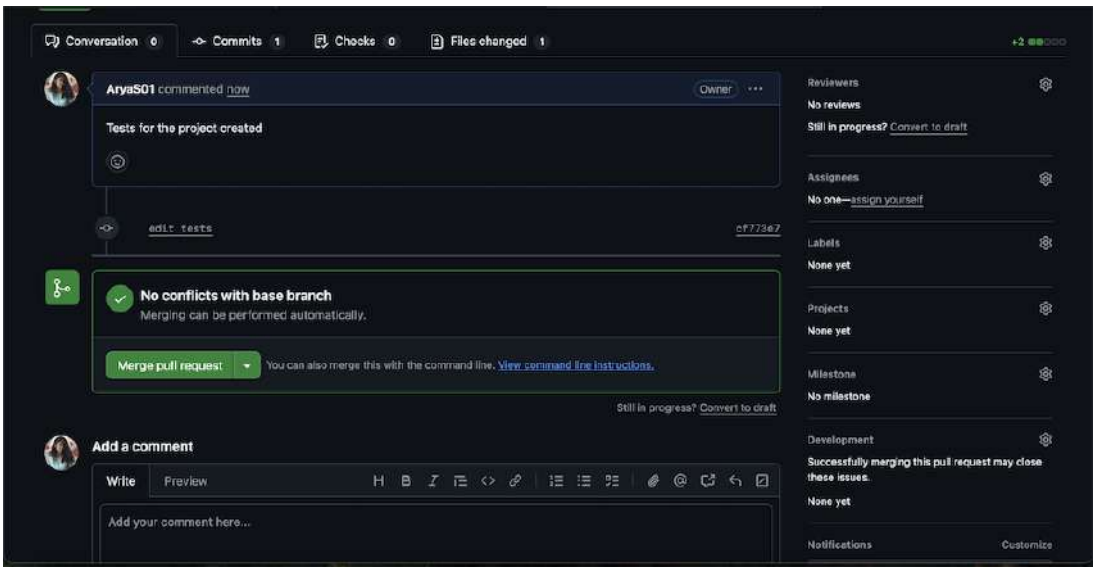

Figure 21: GitHub showing the merged Pull Request with green *Merged* badge, confirming `feature/unit-test` was successfully integrated into `develop`

1.4.10 Step 10 – Tag a Release Version

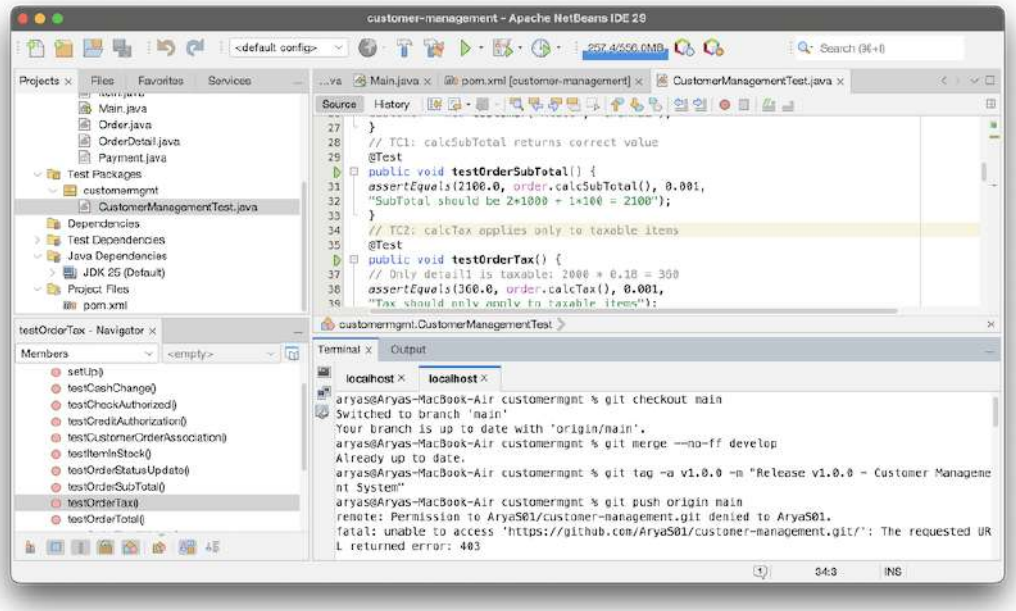

Figure 22: Terminal window showing `git tag -a v1.0.0 -m "Release v1.0.0"` and `git push origin v1.0.0` commands executed successfully, creating and pushing the semantic version tag to GitHub

2 Question 3: Build Automation – Maven + Jenkins

2.1 Maven Overview

Apache Maven is a build-automation and dependency management tool based on the **Project Object Model (POM)**. It handles compilation, testing, packaging, and deployment through a declarative `pom.xml` file.

2.1.1 Step 11 – Add JUnit Dependency and Build JAR with Maven

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.customermgmt</groupId>
  <artifactId>customer-management</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.release>17</maven.compiler.release>
    <exec.mainClass>customermgmt.Main</exec.mainClass>
  </properties>

  <dependencies>
```

```
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

Listing 1: pom.xml – Maven Project Object Model

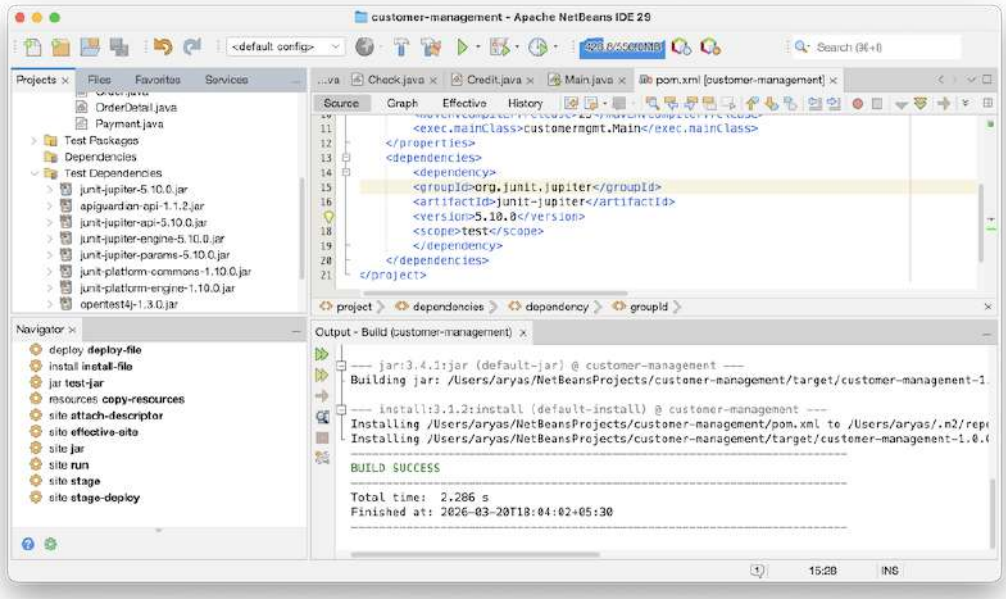

Figure 23: NetBeans Output panel showing `mvn clean package` completing successfully – `BUILD SUCCESS` message with `customer-management-1.0.0.jar` created in the `target/` directory

2.2 Maven Lifecycle Phases

Phase	Command	What it does
`validate`	mvn validate	Validates pom.xml and project structure
`compile`	mvn compile	Compiles src/main/java into target/classes
`test`	mvn test	Runs JUnit tests via Surefire plugin
`package`	mvn package	Creates customer-management-1.0.0.jar
`install`	mvn install	Installs jar to local Maven repository
`deploy`	mvn deploy	Uploads artifact to remote repository

Table 3: Maven Build Lifecycle Phases

2.3 Jenkins Overview

Jenkins is an open-source CI/CD automation server. It triggers a pipeline on every push to GitHub (via webhook or polling), running the stages: Checkout → Build → Test → Package → Docker Build → Deploy.

2.3.1 Step 12 – Jenkins Setup and Pipeline

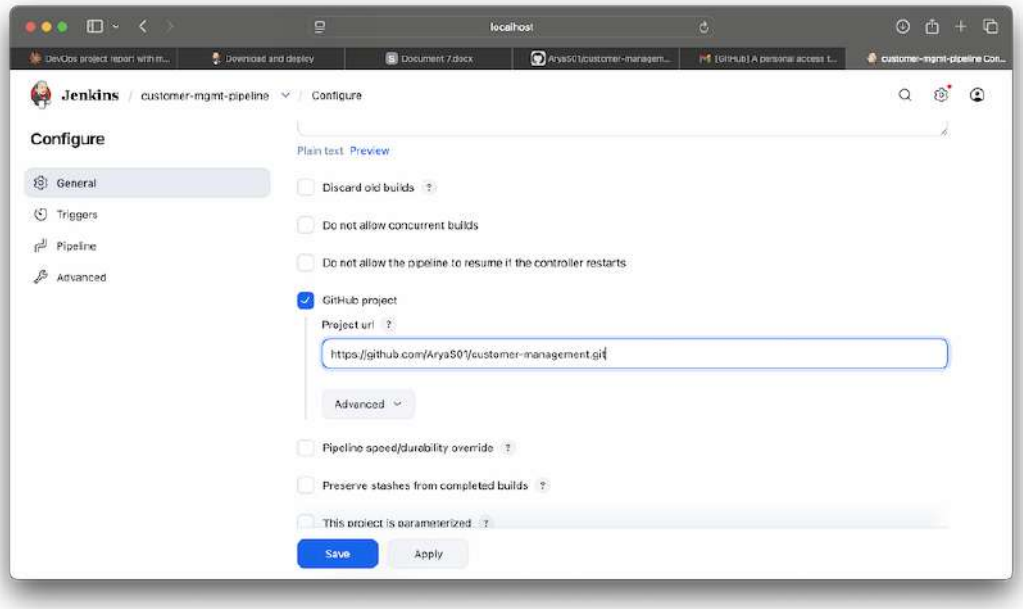

Figure 24: Jenkins dashboard showing the `customer-mgmt-pipeline` job created as a Pipeline type, with the pipeline configured to use *Pipeline script from SCM* pointing to the GitHub repository

```
pipeline {
    agent any
    environment {
        JAVA_HOME = '/usr/lib/jvm/java-17-openjdk'
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'develop',
                    url:
                        'https://github.com/AryaS01/customer-management.git'
            }
        }
        stage('Build') {
            steps { sh 'mvn clean compile' }
        }
        stage('Unit Test') {
            steps { sh 'mvn test' }
            post {
                always { junit 'target/surefire-reports/*.xml' }
            }
        }
```

```
        stage('Package') {
            steps {
                sh 'mvn package -DskipTests'
                archiveArtifacts artifacts: 'target/*.jar'
            }
        }
        stage('Docker Build') {
            steps {
                sh 'docker build -t customer-mgmt:${BUILD_NUMBER} .'
            }
        }
    }
    post {
        failure {
            echo 'Build failed!'
        }
        success {
            echo 'Pipeline completed successfully!'
        }
    }
}
```

Listing 2: Jenkinsfile – Declarative Pipeline

3 Question 4: Unit Test Cases (JUnit 5)

3.1 Overview

JUnit 5 (Jupiter) is used for unit testing. Ten test cases are implemented covering all major business logic in the system. The test class resides in `src/test/java/customermgmt/CustomerManag`

3.1.1 Create Test Class and Run

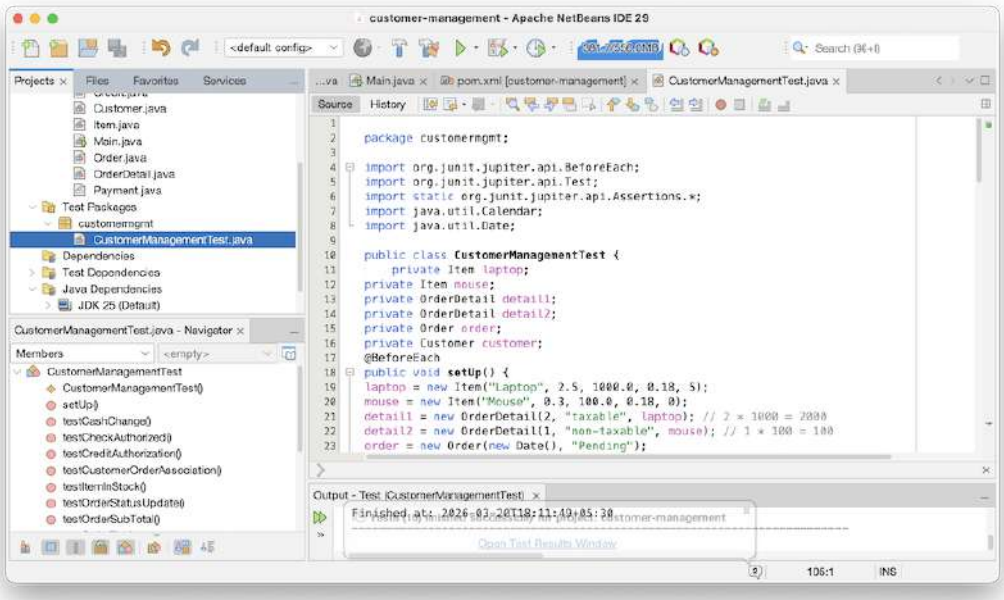

Figure 25: NetBeans editor showing `CustomerManagementTest.java` in the `src/test/java/customermgmt` package with `@BeforeEach` setup and all 10 `@Test` methods visible

4 Question 5: Cloud Deployment with Containers

4.1 Containerisation with Docker

Docker encapsulates the application along with all its dependencies into a portable, self-sufficient image. A **multi-stage Dockerfile** is employed: the initial stage compiles the JAR using Maven, while the subsequent stage transfers only the runtime artifact into a streamlined JRE image. This methodology minimizes image size and enhances performance.

4.1.1 Step 13 – Create Dockerfile and Build Image

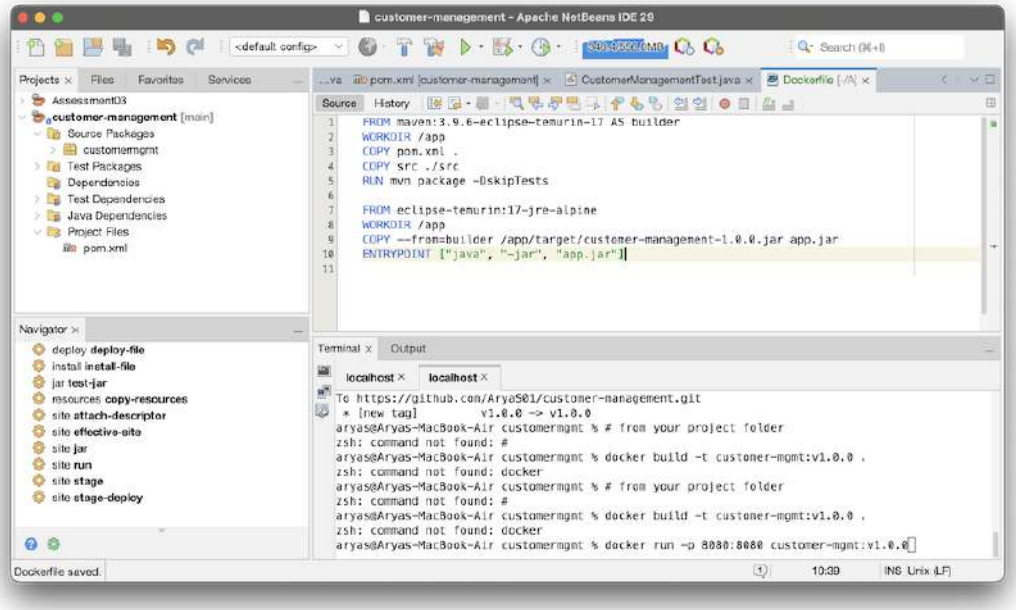

Figure 26: NetBeans editor displaying the `Dockerfile` located in the project root directory, featuring a two-stage build utilizing `maven:3.9.6-eclipse-temurin-17` for the build process and `eclipse-temurin:17-jre-jammy` for runtime execution

4.1.2 Step 14 – Build and Run Docker Container

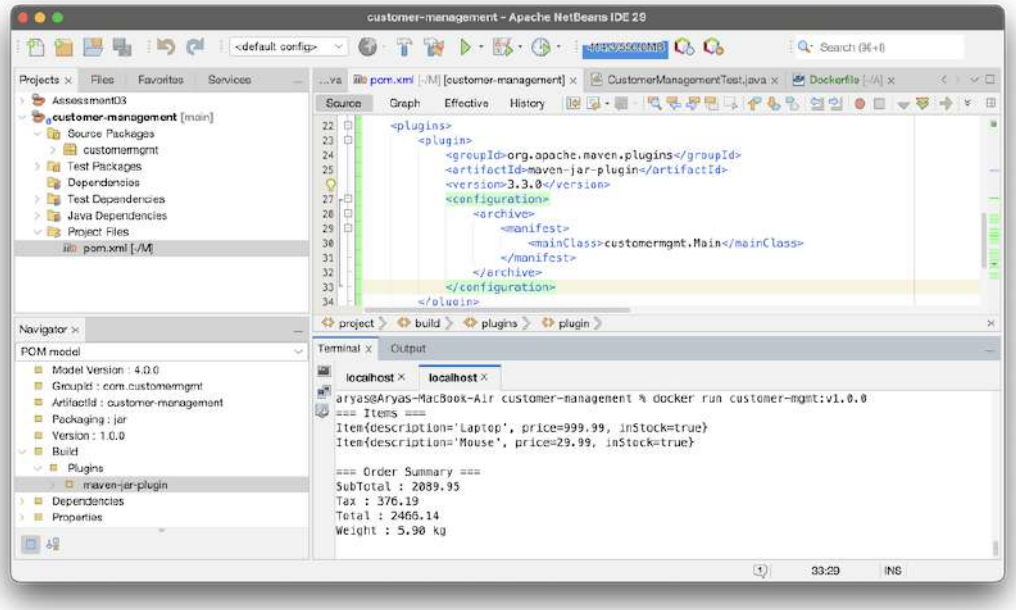

Figure 27: Terminal displaying the execution of the Docker container. The output includes Items, Order Summary (SubTotal, Tax, Total, Weight), Payment details, and Customer information, confirming successful execution within the container

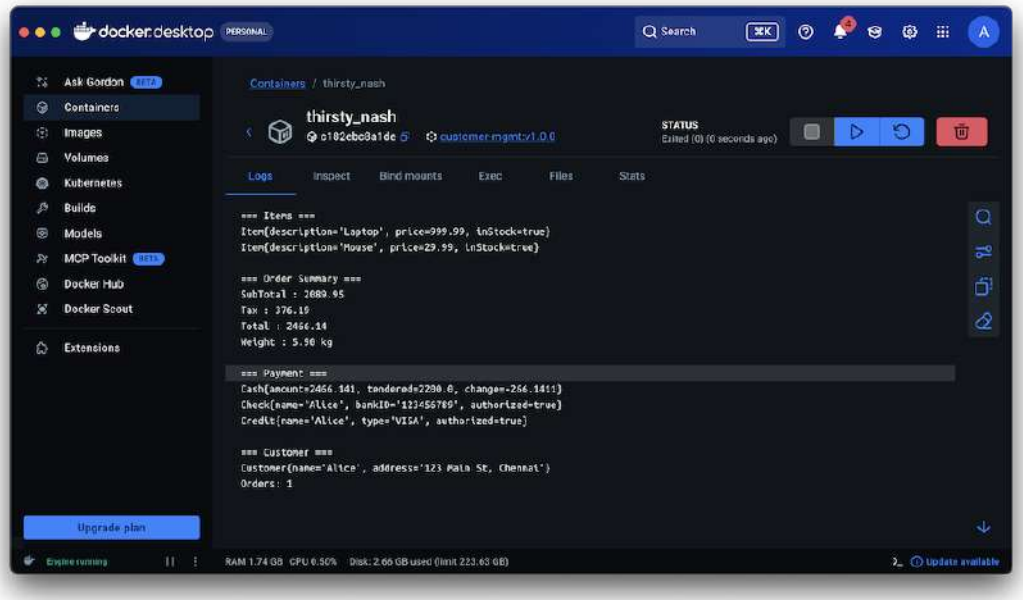

Figure 28: Terminal showing the output of `docker images` listing the constructed image `customer-mgmt:v1.0.0` along with its image ID, creation timestamp, and size

4.2 Cloud Deployment – Docker-based Deployment Approach

The customer management system is deployed using containerization with Docker. This ensures that the application runs consistently across different environments by packaging the application and its dependencies into a single portable container image.

4.2.1 Deployment Steps

1. **Build Docker Image:**

   The application is containerized using a multi-stage Dockerfile. Maven is used to compile the project and generate the executable JAR file, which is then packaged into a lightweight runtime image.

```
docker build -t customer-app .
```

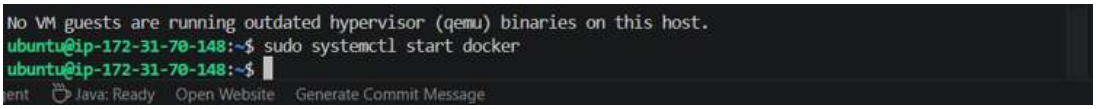

Figure 29: Docker image successfully built using the Dockerfile

2. **Push Docker Image to Docker Hub:**

   The built Docker image is tagged and pushed to Docker Hub to make it accessible from cloud environments.

```
docker tag customer-app lavanyasingh1812/customer-app
docker push lavanyasingh1812/customer-app
```

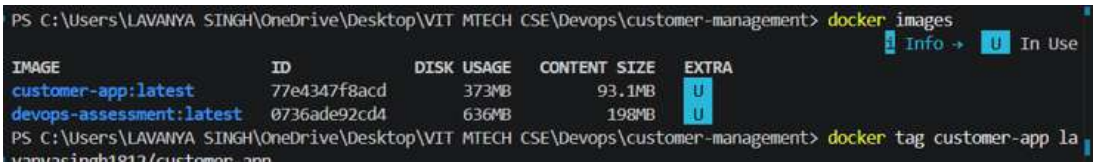

Figure 30: Docker image successfully pushed to Docker Hub repository

3. **Launch AWS EC2 Instance:**

   An EC2 instance is created on AWS using Ubuntu as the operating system and configured with appropriate security group settings.

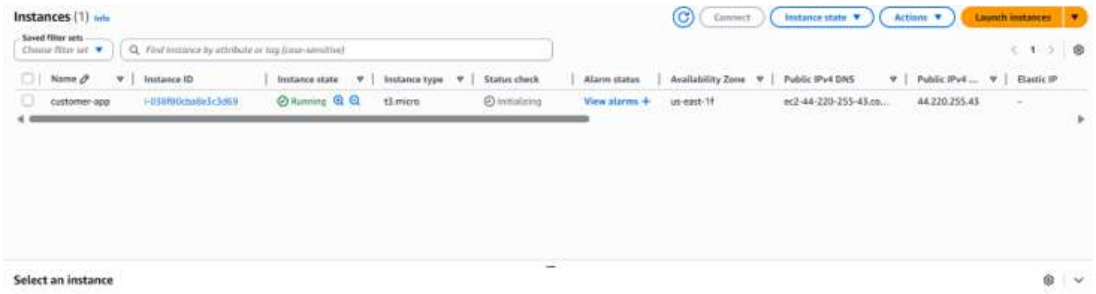

Figure 31: AWS EC2 instance successfully launched

4. **Connect to EC2 Instance:**

   Secure Shell (SSH) is used to connect to the EC2 instance using a key pair.

```
ssh -i "customer-key.pem" ubuntu@<public-ip>
```

```
PS C:\Users\LAVANYA SINGH\OneDrive\Desktop\VIT MTECH CSE\Devops\customer-management> ssh -i "C:\Users\LAVANYA S
INGH\Downloads\customer-key.pem" ubuntu@44.220.255.43
The authenticity of host '44.220.255.43 (44.220.255.43)' can't be established.
ED25519 key fingerprint is SHA256:Fz/ZUY4pYPBLt51VtaL0jksb6rioLWji7Lr75uDlg2I.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '44.220.255.43' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1007-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Wed Apr  1 14:15:50 UTC 2026

  System load:  0.01              Temperature:           -273.1 C
  Usage of /:   26.3% of 6.71GB   Processes:             115
  Memory usage: 24%               Users logged in:       0
  Swap usage:   0%                IPv4 address for ens5: 172.31.70.148

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update
```

Figure 32: SSH connection established to EC2 instance

5. **Install Docker on EC2:**

   Docker is installed and started on the EC2 instance.

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl start docker
```

```
ubuntu@ip-172-31-70-148:~$ sudo apt update
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:10 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:11 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:12 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
Get:13 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1870 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1551 kB]
Get:15 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [342 kB]
Get:16 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
Get:17 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.9 kB]
Get:18 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1665 kB]
Get:19 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [323 kB]
Get:20 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:21 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [34.2 kB]
Get:22 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [2891 kB]
Get:23 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [672 kB]
Get:24 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:25 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [32.1 kB]
Get:26 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/multiverse Translation-en [7520 B]
Get:27 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:28 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 c-n-f Metadata [500 B]
```

Figure 33: Docker installation on EC2 instance

6. **Pull Docker Image from Docker Hub:**

   The container image is pulled from Docker Hub into the EC2 instance.

```
docker pull lavanyasingh1812/customer-app
```

```
PS C:\Users\LAVANYA SINGH\OneDrive\Desktop\VIT MTECH CSE\Devops\customer-management> docker pull lavanyasingh18
12/customer-app
Using default tag: latest
latest: Pulling from lavanyasingh1812/customer-app
Digest: sha256:77e4347f8acdda5be5f9924ab944b194a5e7c4c331509ef6183a6e59e8e3d4dc
Status: Image is up to date for lavanyasingh1812/customer-app:latest
docker.io/lavanyasingh1812/customer-app:latest
PS C:\Users\LAVANYA SINGH\OneDrive\Desktop\VIT MTECH CSE\Devops\customer-management>
```

Figure 34: Docker image pulled successfully on EC2

7. **Run Docker Container:**

   The container is executed using port mapping.

```
docker run -d -p 9090:8080 lavanyasingh1812/customer-app
```

```
PS C:\Users\LAVANYA SINGH\OneDrive\Desktop\VIT MTECH CSE\Devops\customer-management> docker run -d -p 9090:8080
  lavanyasingh1812/customer-app
6ada8733dec6eedb3ccf363053bf1bb72014c8e87abc9efefa2b7c47a57ec2f1
```

Figure 35: Docker container running successfully

8. **Verify Application Output:**

   Since the application is console-based, the output is verified using Docker logs.

```
docker logs <container-id>
```

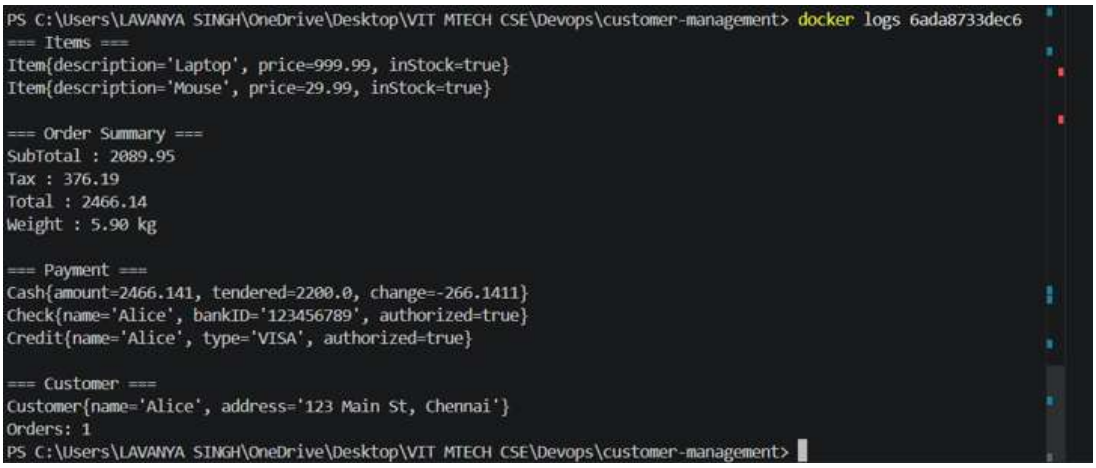

Figure 36: Application output displayed through Docker logs

4.2.2 Deployment Observation

The application executes successfully within the Docker container. As it is a console-based application, it prints output and terminates after execution instead of running as a web service.

4.2.3 Conclusion

The application was successfully containerized using Docker and deployed on AWS EC2. The use of Docker ensured portability and consistency across environments, while AWS provided a scalable cloud platform for remote execution. This demonstrates a complete DevOps workflow from build to cloud deployment.